# 图论竞赛板子

本板子默认 C++17、点编号从 `1` 到 `n`。代码追求赛场可直接复制：变量名较短，接口统一，权值、距离、容量、费用一律使用 `ll`。

## 0. 公共约定与算法选择

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using u64 = uint64_t;

const ll INF = (1LL << 62);
```

`INF` 留出了加法余量。做松弛时仍应先判断距离不是 `INF`，避免无穷大参与运算。

| 问题 | 条件 | 算法 | 复杂度 |
| --- | --- | --- | --- |
| 单源最短路 | 非负边权 | Dijkstra | `O((n+m)log n)` |
| 单源最短路 | 边权只有 0/1 | 0-1 BFS | `O(n+m)` |
| 单源最短路/负环 | 有负边 | Bellman-Ford | `O(nm)` |
| 全源最短路 | `n` 较小 | Floyd | `O(n^3)` |
| 全源最短路 | 稀疏图、有负边无负环 | Johnson | `O(nm+n(m+n)log n)` |
| 最短/次短路计数 | 严格正权 | 双距离 Dijkstra | `O((n+m)log n)` |
| DAG 最长路 | 有向无环图 | 拓扑序 DP | `O(n+m)` |
| 最小生成树 | 无向图 | Kruskal | `O(m log m)` |
| 最小生成树 | 稠密无向图 | Prim | `O(n^2)` |
| 最小树形图 | 有向图、指定根 | Chu-Liu/Edmonds | `O(nm)` |
| 强连通分量 | 有向图 | Tarjan SCC | `O(n+m)` |
| 桥、割点 | 无向图 | Tarjan low-link | `O(n+m)` |
| 二分图判定 | 无向图 | BFS/DFS 染色 | `O(n+m)` |
| 边双/点双分量 | 无向图 | Tarjan | `O(n+m)` |

| 问题 | 条件 | 算法 | 复杂度 |
| --- | --- | --- | --- |
| 最近公共祖先 | 静态树、多次询问 | 倍增 LCA | 预处理 `O(n log n)`，单次 `O(log n)` |
| 少量关键点上的树形 DP | 虚树 | 单次 `O(k log k+k log n)` | |
| 动态最近标记点 | 点分治 | 构建 `O(n log n)`，操作 `O(log^2 n)` | |
| 无根树同构 | 要求确定性 | AHU 精确编码 | 约 `O(n log^2 n)` |
| 有根/无根树快速判同构 | 允许极低碰撞概率 | 双 64 位树哈希 | `O(n)` |
| 欧拉路 | 每条边恰走一次 | Hierholzer | `O(n+m)` |
| 二分图最大匹配 | 无权二分图 | Hopcroft-Karp | `O(m sqrt n)` |
| 一般图最大匹配 | 无权一般图 | Blossom | `O(n^3)` |
| 二分图最大权完美匹配 | 完备权值矩阵 | KM | `O(n^3)` |
| 最大流/最小割 | 容量网络 | Dinic | 一般很快，最坏 `O(n^2m)` |
| 最小费用最大流 | 带费用容量网络 | 势能 Dijkstra | `O(Fm log n)` |
| 2-SAT | 每个约束至多两个布尔变量 | SCC | `O(n+m)` |
| 无向全局最小割 | 无指定源汇 | Stoer-Wagner | `O(n^3)` |
| 标号树编码 | 树与序列互转 | Prüfer | `O(n log n)` |
| 生成树/有向树计数 | 矩阵树定理 | `O(n^3)` | |
| 简单环计数 | `n<=20` 无向图 | 状压 DP | `O(m2^n)` |

# 1. 并查集 DSU

维护无向图的连通块。适合 Kruskal、离线连通性、判环；不支持删除边。

- `DSU d(n)`：建立 `1..n`。
- `find(x)`：返回 `x` 所在集合的代表元。
- `merge(a,b)`：合并两集合；本来已连通返回 `false`，否则返回 `true`。
- `size(x)`：返回 `x` 所在连通块的点数。

路径压缩加按大小合并，均摊复杂度近似 `O(1)`。

```
struct DSU {
    vector<int> p, sz;
    DSU(int n = 0) { init(n); }
    void init(int n) { // n: 点数
        p.resize(n + 1);
        iota(p.begin(), p.end(), 0);
        sz.assign(n + 1, 1);
    }
    int find(int x) { // x: 点编号
        while (x != p[x]) x = p[x] = p[p[x]];
        return x;
    }
    bool merge(int a, int b) { // a,b: 要合并的点
        a = find(a), b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a, sz[a] += sz[b];
        return true;
    }
    int size(int x) { return sz[find(x)]; } // x: 任意点
};
```

---

# 2. 高性能存图：链式前向星

当 `n,m` 很大、只需要遍历边且不需要频繁删除时，链式前向星比 `vector<vector<...>>` 更省常数和内存。

- `Star g(n,m)`：`m` 是有向边数上限；无向边需要按两条边计算。
- `add(u,v,w)`：加入有向边 `u -> v`。
- 遍历 `for (int i=g.h[u]; i!=-1; i=g.e[i].nx)`，终点为 `g.e[i].v`，边权为 `g.e[i].w`。

复杂度和邻接表相同，初始化 `O(n)`，每条边 `O(1)` 加入。

```
struct Star {
    struct E { int v, nx; ll w; };
    int n, tot = 0;
    vector<int> h;
    vector<E> e;
    Star(int n, int m = 0) : n(n), h(n + 1, -1) {
        e.reserve(m);
    }
    void add(int u, int v, ll w = 1) { // u->v, 权 w
        e.push_back({v, h[u], w}), h[u] = tot++;
    }
};
```

若只存无权图，可把 `E::w` 改成 `int` 或删除；本板子保留 `ll` 以便直接复用。

# 3. 最短路

## 3.1 Dijkstra：非负边权

解决非负权图的单源最短路；有向、无向图均可。出现负边时不能使用。

- `Dij dj(n)`：建立 `n` 个点的图。
- `add(u,v,w,bi)`：加入 `u -> v`、权为 `w` 的边；`bi=true` 时同时加入反向边。
- `run(s)`：返回源点 `s` 到所有点的最短距离，无法到达为 `INF`。

复杂度 `O((n+m)log n)`，空间 `O(n+m)`。

```
struct Dij {
    int n;
    vector<vector<pair<int, ll>>> g;
    Dij(int n) : n(n), g(n + 1) {}
    void add(int u, int v, ll w, bool bi = false) { // u,v,w; bi=1 加反边
        g[u].push_back({v, w});
        if (bi) g[v].push_back({u, w});
    }
    vector<ll> run(int s) { // s: 源点
        vector<ll> d(n + 1, INF);
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>>
q;
        d[s] = 0, q.push({0, s});
        while (!q.empty()) {
            auto [dx, u] = q.top(); q.pop();
            if (dx != d[u]) continue;
            for (auto [v, w] : g[u]) {
                if (d[v] > dx + w) {
                    d[v] = dx + w;
                    q.push({d[v], v});
                }
            }
        }
        return d;
    }
};
```

需要还原一条最短路时，在成功松弛处加 `pre[v]=u`，最后从终点沿 `pre` 逆推。

## 3.2 最短路径树 SPT

最短路径树不是最小生成树：给定根 `s`，要求树上 `s` 到每个点的距离都等于原图最短路。Dijkstra 松弛时记录前驱即可得到一棵 SPT。

- `spt(g,s)`：`g` 为非负权邻接表，返回 `{距离,前驱}`。
- 不可达点前驱为 `0`；若有多条最短路，模板保留其中任意一条。
- 复杂度 `O((n+m)log n)`。

```
pair<vector<ll>, vector<int>> spt(const vector<vector<pair<int, ll>>>& g, int s) {
// g: 1-index; s: 根
    int n = (int)g.size() - 1;
    vector<ll> d(n + 1, INF);
    vector<int> pre(n + 1);
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
    d[s] = 0, q.push({0, s});
    while (!q.empty()) {
        auto [du, u] = q.top(); q.pop();
        if (du != d[u]) continue;
        for (auto [v, w] : g[u]) if (d[v] > du + w)
            d[v] = du + w, pre[v] = u, q.push({d[v], v});
    }
    return {move(d), move(pre)};
}
```

## 3.3 0-1 BFS：边权只有 0/1

比 Dijkstra 更快。权为 `0` 的新状态放队首，权为 `1` 的放队尾。

- 参数 `g[u]` 中每项为 `{v,w}`，且 `w` 必须是 `0` 或 `1`。
- `bfs01(g,s)` 返回 `s` 到各点最短距离，点编号默认从 `1` 开始。

复杂度 `O(n+m)`。

```
vector<ll> bfs01(const vector<vector<pair<int, int>>>& g, int s) { // g: 0/1 图; s: 源点
    int n = (int)g.size() - 1;
    vector<ll> d(n + 1, INF);
    vector<char> vis(n + 1);
    deque<int> q;
    d[s] = 0, q.push_front(s);
    while (!q.empty()) {
        int u = q.front(); q.pop_front();
        if (vis[u]) continue;
        vis[u] = 1;
        for (auto [v, w] : g[u]) {
            if (!vis[v] && d[v] > d[u] + w) {
                d[v] = d[u] + w;
                if (w) q.push_back(v);
                else q.push_front(v);
            }
        }
    }
    return d;
}
```

`vis` 保证每个点只展开一次；双端队列始终按非降距离取点，因此首次出队即为最终最短距离，整体为线性复杂度。

## 3.4 Bellman-Ford：负边与可达负环

允许负边。`run(s,d)` 求从 `s` 出发的最短路，同时判断从 `s` 是否能到达负环。

- `add(u,v,w)`：加入有向边。
- `run(s,d)`：把距离写入 `d`；若存在从 `s` 可达的负环返回 `false`，否则返回 `true`。
- 若要判断图中任意位置是否有负环，将全部 `d[i]` 初始化为 `0`，再执行相同的 `n` 轮松弛。

复杂度 `O(nm)`，适合规模较小或负边场景。普通最短路优先用 Dijkstra，不要无条件使用 SPFA。

```
struct BF {
    int n;
    vector<tuple<int, int, ll>> e;
    BF(int n) : n(n) {}
    void add(int u, int v, ll w) { e.push_back({u, v, w}); } // u->v, 权 w
    bool run(int s, vector<ll>& d) { // s: 源点; d: 返回距离
        d.assign(n + 1, INF);
        d[s] = 0;
        for (int i = 1; i <= n; ++i) {
            bool ok = false;
            for (auto [u, v, w] : e) {
                if (d[u] != INF && d[v] > d[u] + w) {
                    d[v] = max(-INF, d[u] + w);
                    ok = true;
                    if (i == n) return false;
                }
            }
            if (!ok) break;
        }
        return true;
    }
};
```

## 3.5 SPFA：快速判负环

只在需要判负环、数据通常较弱时使用。`s=0` 表示从所有点同时出发，判断图中任意负环；传入具体 `s` 只判断从 `s` 可达的负环。

```
bool spfa_neg(const vector<vector<pair<int, ll>>>& g, int s = 0) { // g: 1-index;
s=0 判全图
    int n = (int)g.size() - 1;
    vector<ll> d(n + 1, INF);
    vector<int> cnt(n + 1);
    vector<char> in(n + 1);
    queue<int> q;
    if (!s) {
        fill(d.begin() + 1, d.end(), 0);
        for (int i = 1; i <= n; ++i) q.push(i), in[i] = 1;
    } else d[s] = 0, q.push(s), in[s] = 1;
    while (!q.empty()) {
```

```
        int u = q.front(); q.pop();
        in[u] = 0;
        for (auto [v, w] : g[u]) if (d[v] > d[u] + w) {
            d[v] = d[u] + w, cnt[v] = cnt[u] + 1;
            if (cnt[v] >= n) return true;
            if (!in[v]) in[v] = 1, q.push(v);
        }
    }
    return false;
}
```

SPFA 最坏复杂度仍为 `O(nm)`，有专门构造可以退化；非负边最短路请使用 Dijkstra。

## 3.6 Floyd：全源最短路

适合 `n` 较小的稠密图，也可用于传递闭包、最小环等问题。

- `d` 是 `(n+1)*(n+1)` 邻接矩阵。
- 初始化 `d[i][i]=0`，没有边为 `INF`；重边取最小值。
- 调用后 `d[i][j]` 为最短距离。

复杂度 `O(n^3)`，空间 `O(n^2)`。

```
void floyd(vector<vector<ll>>& d) { // d: 邻接矩阵，原地更新
    int n = (int)d.size() - 1;
    for (int k = 1; k <= n; ++k)
        for (int i = 1; i <= n; ++i) if (d[i][k] != INF)
            for (int j = 1; j <= n; ++j) if (d[k][j] != INF)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}
```

若执行完存在 `d[i][i] < 0`，说明 `i` 能走到某个负环再返回。

## 3.7 最短路与次短路计数

适用于边权严格为正的图。返回每个点的最短距离、次短距离，以及达到这两个距离的路径条数；若题目要求取模，在每次加法处加 `% mod`。

```
struct D2 {
    vector<array<ll, 2>> d, c;
    // g: 1-index 正权图; s: 源点; d[*][0/1] 为最短/次短
    D2(const vector<vector<pair<int, ll>>>& g, int s) {
        int n = (int)g.size() - 1;
        d.resize(n + 1), c.resize(n + 1);
        for (int i = 1; i <= n; ++i) d[i] = {INF, INF}, c[i] = {0, 0};
        priority_queue<tuple<ll, int, int>, vector<tuple<ll, int, int>>,
greater<tuple<ll, int, int>>> q;
        d[s][0] = 0, c[s][0] = 1, q.push({0, s, 0});
        while (!q.empty()) {
            auto [du, u, k] = q.top(); q.pop();
```

```
            if (du != d[u][k]) continue;
            for (auto [v, w] : g[u]) {
                ll x = du + w;
                if (x < d[v][0]) {
                    d[v][1] = d[v][0], c[v][1] = c[v][0];
                    if (d[v][1] != INF) q.push({d[v][1], v, 1});
                    d[v][0] = x, c[v][0] = c[u][k], q.push({x, v, 0});
                } else if (x == d[v][0]) c[v][0] += c[u][k];
                else if (x < d[v][1]) d[v][1] = x, c[v][1] = c[u][k], q.push({x, v,
1});
                else if (x == d[v][1]) c[v][1] += c[u][k];
            }
        }
    }
};
```

存在零权边时可能有无限条等长路径，必须先结合题目对“路径/走法”的定义处理环，不能直接套此计数模板。

## 3.8 Johnson：稀疏图全源最短路

Johnson 允许负边，但不允许负环。先用 Bellman-Ford 求势能 `h`，将每条边重标为 `w+h[u]-h[v]`，新边权均非负，再从每个点跑 Dijkstra。最终还原为 `d[u][v]-h[u]+h[v]`。

- `Johnson j(n)`：建立 `1..n` 的有向图。
- `add(u,v,w)`：加入权为 `w` 的有向边。
- `all()`：返回 `{ok,d}`；`ok=false` 表示存在负环，否则 `d[s][v]` 是最短距离，不可达为 `INF`。

时间复杂度 $O(nm + n(m+n)\log n)$，空间 $O(n^2+m)$。只求单源时直接用 Bellman-Ford，不要抄本段；`n` 很小或图很密时 Floyd 更短。

```
struct Johnson {
    struct E { int u, v; ll w; };
    int n;
    vector<E> e;
    vector<vector<pair<int, ll>>> g;

    Johnson(int n): n(n), g(n + 1) {}
    void add(int u, int v, ll w) { // u：起点；v：终点；w：边权
        e.push_back({u, v, w});
        g[u].push_back({v, w});
    }

    pair<bool, vector<vector<ll>>> all() { // 返回是否无负环及全源距离
        vector<ll> h(n + 1);
        for (int k = 1; k <= n; k++) {
            bool ch = false;
            for (auto [u, v, w] : e) if (h[v] > h[u] + w)
                h[v] = h[u] + w, ch = true;
```

```
            if (!ch) break;
            if (k == n) return {false, {}};
        }

        vector<vector<ll>> ans(n + 1, vector<ll>(n + 1, INF));
        for (int s = 1; s <= n; s++) {
            vector<ll> d(n + 1, INF);
            priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll,
int>>> q;
            d[s] = 0, q.push({0, s});
            while (!q.empty()) {
                auto [du, u] = q.top(); q.pop();
                if (du != d[u]) continue;
                for (auto [v, w] : g[u]) {
                    ll nw = w + h[u] - h[v];
                    if (d[v] > du + nw)
                        d[v] = du + nw, q.push({d[v], v});
                }
            }
            for (int v = 1; v <= n; v++) if (d[v] != INF)
                ans[s][v] = d[v] - h[s] + h[v];
        }
        return {true, ans};
    }
};
```

# 4. 拓扑排序与 DAG 最长路

只适用于有向无环图。`topo()` 返回一个拓扑序；返回长度小于 `n` 表示原图有环。`longest(s)` 求从 `s` 出发的最长路，无法到达的点为 `-INF`。

- `add(u,v,w)`：加入有向边 `u -> v`。
- `topo()`：判环或获取拓扑序。
- `longest(s)`：边权可正可负，但图必须是 DAG。

复杂度 `O(n+m)`。

```
struct DAG {
    int n;
    vector<vector<pair<int, ll>>> g;
    vector<int> in;
    DAG(int n) : n(n), g(n + 1), in(n + 1) {}
    void add(int u, int v, ll w = 0) { // u->v, 权 w
        g[u].push_back({v, w}), ++in[v];
    }
    vector<int> topo() { // 返回拓扑序；不足 n 个说明有环
        vector<int> d = in, o;
        queue<int> q;
        for (int i = 1; i <= n; ++i) if (!d[i]) q.push(i);
        while (!q.empty()) {
```

```
            int u = q.front(); q.pop();
            o.push_back(u);
            for (auto [v, w] : g[u]) if (!--d[v]) q.push(v);
        }
        return o;
    }
    vector<ll> longest(int s) { // s: 起点；图必须为 DAG
        vector<int> o = topo();
        vector<ll> d(n + 1, -INF);
        d[s] = 0;
        if ((int)o.size() != n) return {};
        for (int u : o) if (d[u] != -INF)
            for (auto [v, w] : g[u]) d[v] = max(d[v], d[u] + w);
        return d;
    }
};
```

无权 DAG 的最长路径点数可令每条边权为 `1`，所得是边数，再按题意决定是否加 `1`。

---

# 5. 最小生成树 Kruskal

解决无向带权图的最小生成树或最小生成森林。边权允许为负。

- `e` 中每条边是 `{w,u,v}`。
- `kruskal(n,e,used)` 返回最小生成树边权和。
- 图不连通时返回 `INF`；`used` 可省略，传入时返回所选边的下标。

复杂度 `O(m log m)`，主要是排序。

```
struct KE { int u, v, id; ll w; };

ll kruskal(int n, vector<KE> e, vector<int>* used = nullptr) { // n: 点数; e: 无向边
    sort(e.begin(), e.end(), [](const KE& a, const KE& b) { return a.w < b.w; });
    DSU d(n);
    ll ans = 0;
    int cnt = 0;
    if (used) used->clear();
    for (auto [u, v, id, w] : e) {
        if (!d.merge(u, v)) continue;
        ans += w, ++cnt;
        if (used) used->push_back(id);
        if (cnt == n - 1) break;
    }
    return cnt == n - 1 ? ans : INF;
}
```

若只要求生成森林，删除最后的连通性判断并直接返回 `ans`。判定一条边能否出现在某棵 MST、求严格次小生成树等问题，不能只靠这一份基础模板。

---

## 5.1 Prim：稠密图最小生成树

适合邻接矩阵或接近完全图的无向带权图。与 Kruskal 一样，图不连通时返回 `INF`。

- `a[u][v]`：边权矩阵；无边填 `INF`，对角线填 `0`，重边取最小值。
- `prim(a, pre)`：返回 MST 权值；`pre` 可选，用来得到每个点在 MST 中的父点。

复杂度 `O(n^2)`，空间 `O(n^2)`。稀疏图优先使用 Kruskal 或优先队列 Prim。

```
ll prim(const vector<vector<ll>>& a, vector<int>* pre = nullptr) { // a: 1-index 权值矩阵
    int n = (int)a.size() - 1;
    vector<ll> d(n + 1, INF);
    vector<char> vis(n + 1);
    if (pre) pre->assign(n + 1, 0);
    d[1] = 0;
    ll ans = 0;
    for (int k = 1; k <= n; ++k) {
        int u = -1;
        for (int i = 1; i <= n; ++i)
            if (!vis[i] && (u == -1 || d[i] < d[u])) u = i;
        if (u == -1 || d[u] == INF) return INF;
        vis[u] = 1, ans += d[u];
        for (int v = 1; v <= n; ++v) if (!vis[v] && a[u][v] < d[v]) {
            d[v] = a[u][v];
            if (pre) (*pre)[v] = u;
        }
    }
    return ans;
}
```

## 5.2 Chu-Liu/Edmonds：有向最小树形图

求从指定根 `rt` 出发、能到达所有点的最小有向生成树（最小树形图）。边方向是父节点指向子节点，允许负边，支持重边。

- `DEdge{u,v,w}`：有向边 `u -> v`，权值 `w`。
- `dmst(n,rt,e)`：返回最小树形图权值；若根无法到达全部点返回 `INF`。
- 点编号使用 `1..n`。

复杂度 `O(nm)`，适合中小规模；这是有向图版本，不要与无向 MST 混用。

```
struct DEdge { int u, v; ll w; };

ll dmst(int n, int rt, vector<DEdge> e) { // n: 点数; rt: 根; e: 有向边
    ll ans = 0;
    while (1) {
        vector<ll> in(n + 1, INF);
        vector<int> pre(n + 1, -1), id(n + 1, -1), vis(n + 1, -1);
        for (auto z : e) if (z.u != z.v && z.w < in[z.v])
```

```
            in[z.v] = z.w, pre[z.v] = z.u;
        in[rt] = 0;
        for (int i = 1; i <= n; ++i) if (in[i] == INF) return INF;
        int cnt = 0;
        for (int i = 1; i <= n; ++i) {
            ans += in[i];
            int v = i;
            while (vis[v] != i && id[v] == -1 && v != rt) vis[v] = i, v = pre[v];
            if (v != rt && id[v] == -1) {
                for (int u = pre[v]; u != v; u = pre[u]) id[u] = cnt + 1;
                id[v] = ++cnt;
            }
        }
        if (!cnt) break;
        for (int i = 1; i <= n; ++i) if (id[i] == -1) id[i] = ++cnt;
        vector<DEdge> ne;
        for (auto z : e) {
            int u = id[z.u], v = id[z.v];
            if (u != v) ne.push_back({u, v, z.w - in[z.v]});
        }
        rt = id[rt], n = cnt, e.swap(ne);
    }
    return ans;
}
```

# 6. 有向图强连通分量 SCC

同一强连通分量内任意两点互相可达。缩点后一定得到 DAG，常用于有向图判环、可达性结构、2-SAT、缩点 DP。

- `SCC s(n)` 后用 `add(u,v)` 加有向边。
- `work()` 返回 SCC 数量。
- `id[u]` 是点 `u` 所属分量编号，范围 `0..cnt-1`。
- `sz[c]` 是分量 `c` 的点数。
- `build()` 返回去重后的缩点 DAG。

复杂度 `O(n+m)`。模板使用递归 DFS，链状图很深时需确保栈空间足够。

```
struct SCC {
    int n, tim = 0, cnt = 0;
    vector<vector<int>> g;
    vector<int> dfn, low, stk, id, sz;
    vector<char> ins;
    SCC(int n) : n(n), g(n + 1), dfn(n + 1), low(n + 1),
                 id(n + 1, -1), ins(n + 1) {}
    void add(int u, int v) { g[u].push_back(v); } // u->v
    void dfs(int u) {
        dfn[u] = low[u] = ++tim;
        stk.push_back(u), ins[u] = 1;
        for (int v : g[u]) {
```

```
            if (!dfn[v]) dfs(v), low[u] = min(low[u], low[v]);
            else if (ins[v]) low[u] = min(low[u], dfn[v]);
        }
        if (dfn[u] != low[u]) return;
        sz.push_back(0);
        while (1) {
            int v = stk.back(); stk.pop_back();
            ins[v] = 0, id[v] = cnt, ++sz[cnt];
            if (v == u) break;
        }
        ++cnt;
    }
    int work() { // 返回 SCC 数
        for (int i = 1; i <= n; ++i) if (!dfn[i]) dfs(i);
        return cnt;
    }
    vector<vector<int>> build() { // work 后调用；返回缩点 DAG
        vector<vector<int>> dag(cnt);
        for (int u = 1; u <= n; ++u)
            for (int v : g[u]) if (id[u] != id[v]) dag[id[u]].push_back(id[v]);
        for (auto& a : dag) {
            sort(a.begin(), a.end());
            a.erase(unique(a.begin(), a.end()), a.end());
        }
        return dag;
    }
};
```

常用结论：若原图不是单个 SCC，缩点 DAG 中入度为 `0` 的分量数为 `a`、出度为 `0` 的分量数为 `b`，使原图变成强连通图至少要添加 `max(a,b)` 条边。

---

# 7. 无向图的桥与割点

桥是删掉后使连通块数增加的边；割点是删掉该点及关联边后使连通块数增加的点。模板按边编号处理，能正确应对重边。

- `add(u,v)`：加入一条无向边，返回边编号 `0..m-1`。
- `work()`：处理所有连通块。
- `bridge[id]` 表示该边是否为桥。
- `cut[u]` 表示点 `u` 是否为割点。

复杂度 `O(n+m)`。自环不会成为桥；平行边不会被误判为桥。

```
struct LowLink {
    int n, tim = 0, m = 0;
    vector<vector<pair<int, int>>> g;
    vector<int> dfn, low;
    vector<char> cut, bridge;
    LowLink(int n) : n(n), g(n + 1), dfn(n + 1), low(n + 1), cut(n + 1) {}
```

```
    int add(int u, int v) { // u-v 无向边；返回边号
        int id = m++;
        g[u].push_back({v, id}), g[v].push_back({u, id});
        bridge.push_back(0);
        return id;
    }
    void dfs(int u, int pe) {
        dfn[u] = low[u] = ++tim;
        int son = 0;
        for (auto [v, id] : g[u]) {
            if (id == pe) continue;
            if (!dfn[v]) {
                ++son, dfs(v, id);
                low[u] = min(low[u], low[v]);
                if (low[v] > dfn[u]) bridge[id] = 1;
                if (pe != -1 && low[v] >= dfn[u]) cut[u] = 1;
            } else low[u] = min(low[u], dfn[v]);
        }
        if (pe == -1 && son > 1) cut[u] = 1;
    }
    void work() { // 处理整张图
        for (int i = 1; i <= n; ++i) if (!dfn[i]) dfs(i, -1);
    }
};
```

删掉所有桥后，每个连通块是一个边双连通分量。把这些分量缩点后得到森林；原图连通时得到一棵树。若要添加最少边使一个连通无向图变成边双连通图，设缩点树叶子数为 `k`，答案为 `(k+1)/2`。

---

# 8. 边双连通分量与点双连通分量

## 8.1 边双连通分量 E-DCC

删掉所有桥后剩余的连通块就是边双连通分量。`work()` 后 `id[u]` 为点 `u` 所属分量，`cc[c]` 是第 `c` 个分量的点集，`bridge` 记录桥；`tr` 是 `{相邻分量, 原边号}` 形式的桥森林。

```
struct EDCC {
    int n, m = 0, tim = 0, cnt = 0;
    vector<vector<pair<int, int>>> g;
    vector<int> eu, ev, dfn, low, id;
    vector<char> bridge;
    vector<vector<int>> cc;
    vector<vector<pair<int, int>>> tr;
    EDCC(int n) : n(n), g(n + 1), dfn(n + 1), low(n + 1), id(n + 1, -1) {}
    int add(int u, int v) { // u-v 无向边；返回边号
        int z = m++;
        eu.push_back(u), ev.push_back(v);
        g[u].push_back({v, z}), g[v].push_back({u, z});
        bridge.push_back(0);
        return z;
    }
```

```
    void dfs1(int u, int pe) {
        dfn[u] = low[u] = ++tim;
        for (auto [v, z] : g[u]) {
            if (z == pe) continue;
            if (!dfn[v]) {
                dfs1(v, z), low[u] = min(low[u], low[v]);
                if (low[v] > dfn[u]) bridge[z] = 1;
            } else low[u] = min(low[u], dfn[v]);
        }
    }
    void dfs2(int u, int c) {
        id[u] = c;
        cc[c].push_back(u);
        for (auto [v, z] : g[u]) if (!bridge[z] && id[v] == -1) dfs2(v, c);
    }
    int work() { // 返回 E-DCC 数量
        tim = cnt = 0;
        fill(dfn.begin(), dfn.end(), 0), fill(low.begin(), low.end(), 0);
        fill(id.begin(), id.end(), -1), fill(bridge.begin(), bridge.end(), 0);
        for (int i = 1; i <= n; ++i) if (!dfn[i]) dfs1(i, -1);
        cc.clear();
        for (int i = 1; i <= n; ++i) if (id[i] == -1) {
            cc.push_back({});
            dfs2(i, cnt++);
        }
        tr.assign(cnt, {});
        for (int z = 0; z < m; ++z) if (bridge[z]) {
            int a = id[eu[z]], b = id[ev[z]];
            tr[a].push_back({b, z}), tr[b].push_back({a, z});
        }
        return cnt;
    }
};
```

## 8.2 点双连通分量 V-DCC

点双分量是极大的不含割点分离的子图。`bcc` 中每个数组保存一个点双分量的点集；割点可能出现在多个分量中。

- `add(u,v)`：加入无向边并返回边号 `z`，端点由 `eu[z]`、`ev[z]` 访问。
- `work()`：返回所有点双分量，并填充 `bcc`、`ebcc`、`bel` 与 `cut`；`ebcc[c]` 与 `bcc[c]` 对应。
- `build()`：在 `tr` 中建立圆方森林，原点编号 `1..n`，方点编号 `n+c+1`。
- 复杂度 `O(n+m)`；边编号用于正确处理重边。

```
struct VDCC {
    int n, m = 0, tim = 0;
    vector<vector<pair<int, int>>> g;
    vector<int> eu, ev, dfn, low, stk, mk;
    vector<char> cut;
    vector<vector<int>> bcc, ebcc, bel, tr;
```

```
    vector<char> seen;
    int tag = 0;
    VDCC(int n) : n(n), g(n + 1), dfn(n + 1), low(n + 1), mk(n + 1), cut(n + 1) {}
    int add(int u, int v) { // u-v 无向边；返回边号
        int z = m++;
        eu.push_back(u), ev.push_back(v);
        g[u].push_back({v, z}), g[v].push_back({u, z});
        seen.push_back(0);
        return z;
    }
    void take(int stop, int root) {
        ++tag;
        vector<int> c, e;
        while (1) {
            int z = stk.back(); stk.pop_back();
            e.push_back(z);
            if (mk[eu[z]] != tag) mk[eu[z]] = tag, c.push_back(eu[z]);
            if (mk[ev[z]] != tag) mk[ev[z]] = tag, c.push_back(ev[z]);
            if (z == stop) break;
        }
        if (!c.empty()) bcc.push_back(move(c)), ebcc.push_back(move(e));
        (void)root;
    }
    void dfs(int u, int pe) {
        dfn[u] = low[u] = ++tim;
        int son = 0;
        for (auto [v, z] : g[u]) {
            if (z == pe) continue;
            if (u == v) {
                if (!seen[z]) {
                    seen[z] = 1;
                    bcc.push_back({u}), ebcc.push_back({z});
                }
                continue;
            }
            if (!dfn[v]) {
                ++son, seen[z] = 1, stk.push_back(z), dfs(v, z);
                low[u] = min(low[u], low[v]);
                if (low[v] >= dfn[u]) {
                    if (pe != -1 || son > 1) cut[u] = 1;
                    take(z, u);
                }
            } else if (dfn[v] < dfn[u]) {
                if (!seen[z]) seen[z] = 1, stk.push_back(z);
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (pe == -1 && son == 0) bcc.push_back({u}), ebcc.push_back({});
    }
    int work() { // 返回 V-DCC 数量
        tim = tag = 0;
        fill(dfn.begin(), dfn.end(), 0), fill(low.begin(), low.end(), 0);
```

```
        fill(cut.begin(), cut.end(), 0), fill(seen.begin(), seen.end(), 0);
        stk.clear(), bcc.clear(), ebcc.clear();
        for (int i = 1; i <= n; ++i) if (!dfn[i]) dfs(i, -1);
        bel.assign(n + 1, {});
        for (int c = 0; c < (int)bcc.size(); ++c)
            for (int u : bcc[c]) bel[u].push_back(c);
        return (int)bcc.size();
    }
    void build() { // 建圆方森林：原点 1..n，方点 n+c+1
        work();
        tr.assign(n + bcc.size() + 1, {});
        for (int c = 0; c < (int)bcc.size(); ++c) {
            int x = n + c + 1;
            for (int u : bcc[c]) tr[x].push_back(u), tr[u].push_back(x);
        }
    }
};
```

# 9. 二分图判定

无向图可二分染色，当且仅当不存在奇环。`work(g,col)` 处理非连通图；成功返回 `true`，并把颜色写入 `col`（`0/1`）。发现同色相邻点时返回 `false`。

```
bool bip(const vector<vector<int>>& g, vector<int>& col) { // g: 1-index 无向图; col: 输出颜色
    int n = (int)g.size() - 1;
    col.assign(n + 1, -1);
    queue<int> q;
    for (int s = 1; s <= n; ++s) if (col[s] == -1) {
        col[s] = 0, q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : g[u]) {
                if (col[v] == -1) col[v] = col[u] ^ 1, q.push(v);
                else if (col[v] == col[u]) return false;
            }
        }
    }
    return true;
}
```

# 10. 倍增 LCA

用于静态树上的最近公共祖先、两点距离和向上跳祖先。边可以带权。

- `LCA lca(n)`，随后 `add(u,v,w)` 加无向树边。
- `build(rt)`：以 `rt` 为根预处理，必须先调用；同时得到 `tin/tout` DFS 序。
- `lca.get(u,v)`：返回最近公共祖先。
- `lca.dist(u,v)`：返回树上带权距离。

- `lca.jump(u,k)`：从 `u` 向上走 `k` 条边；超出根返回 `0`。
- `lca.is_anc(u,v)`：判断 `u` 是否为 `v` 的祖先。

预处理 `O(n log n)`，每次询问 `O(log n)`，空间 `O(n log n)`。

```
struct LCA {
    int n, lg;
    vector<vector<pair<int, ll>>> g;
    vector<vector<int>> up;
    vector<int> dep, tin, tout;
    vector<ll> ds;
    LCA(int n) : n(n), lg(1) {
        while ((1 << lg) <= n) ++lg;
        g.resize(n + 1);
        up.assign(lg, vector<int>(n + 1));
        dep.assign(n + 1, -1);
        tin.resize(n + 1), tout.resize(n + 1);
        ds.assign(n + 1, 0);
    }
    void add(int u, int v, ll w = 1) { // 树边 u-v，权 w
        g[u].push_back({v, w}), g[v].push_back({u, w});
    }
    void build(int rt = 1) { // rt: 根
        for (auto &v : up) fill(v.begin(), v.end(), 0);
        fill(dep.begin(), dep.end(), -1);
        fill(ds.begin(), ds.end(), 0);
        int tim = 0;
        struct S { int u, p, i; };
        vector<S> st{{rt, 0, 0}};
        dep[rt] = 0;
        while (!st.empty()) {
            S &x = st.back();
            int u = x.u;
            if (!x.i) {
                tin[u] = ++tim, up[0][u] = x.p;
                for (int j = 1; j < lg; j++) up[j][u] = up[j - 1][up[j - 1][u]];
            }
            if (x.i == (int)g[u].size()) {
                tout[u] = tim, st.pop_back();
                continue;
            }
            auto [v, w] = g[u][x.i++];
            if (v == x.p || dep[v] != -1) continue;
            dep[v] = dep[u] + 1, ds[v] = ds[u] + w;
            st.push_back({v, u, 0});
        }
    }
    int jump(int u, int k) { // u 向上跳 k 条边
        if (u <= 0 || k < 0 || k > dep[u]) return 0;
        for (int j = 0; j < lg; ++j) if (k >> j & 1) u = up[j][u];
        return u;
```

```
    }
    int get(int u, int v) { // u,v 的 LCA
        if (dep[u] < dep[v]) swap(u, v);
        u = jump(u, dep[u] - dep[v]);
        if (u == v) return u;
        for (int j = lg - 1; j >= 0; --j)
            if (up[j][u] != up[j][v]) u = up[j][u], v = up[j][v];
        return up[0][u];
    }
    ll dist(int u, int v) { // u,v 的带权距离
        int p = get(u, v);
        return ds[u] + ds[v] - 2 * ds[p];
    }
    bool is_anc(int u, int v) { // u、v：点编号；判断 u 是否为 v 祖先
        return tin[u] <= tin[v] && tout[v] <= tout[u];
    }
};
```

该实现假定输入是一棵连通树。森林需要对每个根分别 BFS，并额外记录所属连通块，跨块询问没有 LCA。

## 10.1 树的直径

边权非负时，从任意点 `s` 找最远点 `a`，再从 `a` 找最远点 `b`，`a-b` 即为一条直径。下面使用迭代 DFS，避免树链过深时递归爆栈。

```
pair<int, ll> tree_far(const vector<vector<pair<int, ll>>>& g, int s) { // g: 1-index 树; s: 起点
    int ba = s;
    ll bd = 0;
    vector<tuple<int, int, ll>> st = {{s, 0, 0}};
    while (!st.empty()) {
        auto [u, fa, d] = st.back(); st.pop_back();
        if (d > bd) bd = d, ba = u;
        for (auto [v, w] : g[u]) if (v != fa) st.push_back({v, u, d + w});
    }
    return {ba, bd};
}

tuple<int, int, ll> tree_diam(const vector<vector<pair<int, ll>>>& g) { // 返回两个端点和长度
    auto [a, _] = tree_far(g, 1);
    auto [b, d] = tree_far(g, a);
    return {a, b, d};
}
```

## 10.2 虚树

一组关键点只关心它们之间的祖先关系时，将关键点和必要的 LCA 压成虚树。虚树边连接的是原树上的祖先和后代，边权取原树距离；节点数不超过 `2k-1`。

本段直接复用上一节增强后的 `LCA`。`build_vt(a,t)` 中 `a` 是本次关键点，`t` 已完成 `build()`；返回的 `v` 包含关键点和新增 LCA，`e` 中每条边为 `{父,子,原树距离}`。空关键点返回空树。

单次复杂度 $O(k\log k + k\log n)$。多次询问时只清空返回的 `v` 中涉及的临时数组位置，不要每次清空 `1..n`。

```
struct VT {
    int rt = 0;
    vector<int> v;
    vector<tuple<int, int, ll>> e;
};

VT build_vt(vector<int> a, LCA &t) { // a：关键点；t：已预处理的 LCA
    VT r;
    if (a.empty()) return r;
    sort(a.begin(), a.end(), [&](int x, int y) { return t.tin[x] < t.tin[y]; });
    a.erase(unique(a.begin(), a.end()), a.end());
    int k = a.size();
    for (int i = 1; i < k; i++) a.push_back(t.get(a[i - 1], a[i]));
    sort(a.begin(), a.end(), [&](int x, int y) { return t.tin[x] < t.tin[y]; });
    a.erase(unique(a.begin(), a.end()), a.end());

    vector<int> st;
    for (int u : a) {
        while (!st.empty() && !t.is_anc(st.back(), u)) st.pop_back();
        if (!st.empty()) r.e.push_back({st.back(), u, t.dist(st.back(), u)});
        st.push_back(u);
    }
    r.rt = a[0], r.v = move(a);
    return r;
}
```

## 10.3 点分治：动态最近标记点

点分治每次找当前连通块的重心，形成高度 $O(\log n)$ 的重心树。下面是一个完整常用版本：支持把点加入/删除标记集合，以及查询任意点到最近标记点的树上距离。

- `Cen c(n)`，用 `add(u,v,w)` 加带非负权树边，再调用 `build()`。
- `on(u)` 加入一个标记，`off(u)` 删除一次对应标记；两者必须配对。
- `ask(u)` 返回到最近标记点的距离；当前没有标记时返回 `INF`。

构建 $O(n\log n)$，空间 $O(n\log n)$；单次修改或查询 $O(\log^2 n)$。若只有加入没有删除，把每个重心的 `multiset` 换成一个最小值即可把操作降为 $O(\log n)$。

```
struct Cen {
```

```
    int n;
    vector<vector<pair<int, ll>>> g;
    vector<int> sz, fa, tmp;
    vector<char> dead;
    vector<vector<pair<int, ll>>> p;
    vector<multiset<ll>> s;

    Cen(int n): n(n), g(n + 1), sz(n + 1), fa(n + 1), tmp(n + 1),
                dead(n + 1), p(n + 1), s(n + 1) {}

    void add(int u, int v, ll w = 1) { // u、v：端点；w：非负边权
        g[u].push_back({v, w}), g[v].push_back({u, w});
    }

    int cen(int rt) { // rt：当前连通块任意点；返回重心
        vector<int> v{rt};
        tmp[rt] = 0;
        for (int i = 0; i < (int)v.size(); i++) {
            int u = v[i];
            for (auto [x, w] : g[u]) if (!dead[x] && x != tmp[u])
                tmp[x] = u, v.push_back(x);
        }
        for (int i = (int)v.size() - 1; i >= 0; i--) {
            int u = v[i]; sz[u] = 1;
            for (auto [x, w] : g[u]) if (!dead[x] && tmp[x] == u) sz[u] += sz[x];
        }
        int c = rt, best = v.size();
        for (int u : v) {
            int mx = (int)v.size() - sz[u];
            for (auto [x, w] : g[u]) if (!dead[x] && tmp[x] == u) mx = max(mx,
sz[x]);
            if (mx < best) best = mx, c = u;
        }
        return c;
    }

    void paths(int c) { // c：本层重心；记录当前分量内各点到 c 的距离
        vector<tuple<int, int, ll>> st{{c, 0, 0}};
        while (!st.empty()) {
            auto [u, f, d] = st.back(); st.pop_back();
            p[u].push_back({c, d});
            for (auto [v, w] : g[u]) if (v != f && !dead[v])
                st.push_back({v, u, d + w});
        }
    }

    void dfs(int rt, int f) { // rt：当前分量入口；f：重心树父亲
        int c = cen(rt);
        fa[c] = f, dead[c] = 1;
        paths(c);
        for (auto [v, w] : g[c]) if (!dead[v]) dfs(v, c);
    }
```

```
    void build(int rt = 1) { // rt：原树任意点
        dfs(rt, 0);
    }
    void on(int u) { // u：加入标记的点
        for (auto [c, d] : p[u]) s[c].insert(d);
    }
    void off(int u) { // u：删除一次标记；要求此前多加入一次
        for (auto [c, d] : p[u]) {
            auto it = s[c].find(d);
            assert(it != s[c].end());
            s[c].erase(it);
        }
    }
    ll ask(int u) { // u：查询点；返回最近标记距离
        ll r = INF;
        for (auto [c, d] : p[u]) if (!s[c].empty()) r = min(r, d + *s[c].begin());
        return r;
    }
};
```

## 10.4 无根树同构：AHU 精确编码

比较两棵无根树是否同构时，先找一个或两个重心，再对有根子树的"子节点编码有序多重集"统一编号。下面使用共享的 `map` 给结构编号，不依赖随机哈希，因此没有碰撞。

`TreeIso t; t.same(a,b)` 判断两棵树是否同构；`a,b` 都是 `1` 开始的无权邻接表。复杂度约 $O(n \log^2 n)$，树为链时也不会递归爆栈。若节点带标签，把标签一并放入 `key` 即可。

```
struct TreeIso {
    map<vector<int>, int> mp;
    int id = 0;

    vector<int> centers(const vector<vector<int>> &g) { // g：1-index 无根树
        int n = (int)g.size() - 1;
        vector<int> d(n + 1);
        queue<int> q;
        for (int i = 1; i <= n; i++) if ((d[i] = g[i].size()) <= 1) q.push(i);
        int left = n;
        while (left > 2) {
            int z = q.size(); left -= z;
            while (z--) {
                int u = q.front(); q.pop();
                for (int v : g[u]) if (--d[v] == 1) q.push(v);
            }
        }
        vector<int> c;
        while (!q.empty()) c.push_back(q.front()), q.pop();
        return c;
    }
```

```
    int root_id(const vector<vector<int>> &g, int rt, int ban) {
        // g：树；rt：根；ban：禁止经过的根相邻点，0 表示无
        int n = (int)g.size() - 1;
        vector<int> f(n + 1, -1), ord{rt}, h(n + 1);
        f[rt] = ban;
        for (int i = 0; i < (int)ord.size(); i++) {
            int u = ord[i];
            for (int v : g[u]) if (v != f[u]) f[v] = u, ord.push_back(v);
        }
        for (int i = (int)ord.size() - 1; i >= 0; i--) {
            int u = ord[i];
            vector<int> key;
            for (int v : g[u]) if (f[v] == u) key.push_back(h[v]);
            sort(key.begin(), key.end());
            auto [it, ok] = mp.emplace(move(key), id + 1);
            if (ok) id++;
            h[u] = it->second;
        }
        return h[rt];
    }

    pair<int, int> code(const vector<vector<int>> &g) { // 返回无根树规范编号对
        auto c = centers(g);
        if (c.size() == 1) return {root_id(g, c[0], 0), 0};
        int a = root_id(g, c[0], c[1]), b = root_id(g, c[1], c[0]);
        if (a > b) swap(a, b);
        return {a, b};
    }

    bool same(const vector<vector<int>> &a, const vector<vector<int>> &b) {
        // a、b：两棵 1-index 无根树
        return a.size() == b.size() && code(a) == code(b);
    }
};
```

## 10.5 高性能树哈希

把有根树编码为“所有儿子哈希的无序多重集”。这里对每个儿子哈希分别随机映射后求和，顺序无关且能保留相同子树的出现次数；使用两组独立的 64 位盐，碰撞概率极低。

- `build(g,rt)`：以 `rt` 为根，返回每个节点的有根子树哈希 `h[u]`。
- `root(g,rt)`：返回以 `rt` 为根的整棵树哈希。
- `code(g)`：返回无根树哈希。先找一个或两个重心；有两个重心时删除它们之间的边，分别哈希两半并排序。
- `same(a,b)`：判断两棵无根树是否同构；`a,b` 都是 `1` 开始的无权树邻接表。

单次时间和空间均为 `O(n)`，全程迭代遍历，不会在链上爆栈。它是概率算法；要求绝对正确时使用上一节 AHU 精确编码。同一个 `TreeHash` 对象内的盐固定，可以连续比较多棵树；哈希值使用运行时随机盐，不能跨进程保存或比较。

```cpp
struct TreeHash {
    using H = array<u64, 2>;
    H sd;

    static u64 rnd(u64 x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }

    TreeHash(u64 s = chrono::steady_clock::now().time_since_epoch().count()) {
        sd = {rnd(s), rnd(s ^ 0x6a09e667f3bcc909ULL)};
    }

    vector<int> cen(const vector<vector<int>> &g) { // g：1-index 无根树
        int n = (int)g.size() - 1;
        vector<int> d(n + 1);
        queue<int> q;
        for (int i = 1; i <= n; i++) if ((d[i] = g[i].size()) <= 1) q.push(i);
        int left = n;
        while (left > 2) {
            int z = q.size(); left -= z;
            while (z--) {
                int u = q.front(); q.pop();
                for (int v : g[u]) if (--d[v] == 1) q.push(v);
            }
        }
        vector<int> c;
        while (!q.empty()) c.push_back(q.front()), q.pop();
        return c;
    }

    vector<H> build(const vector<vector<int>> &g, int rt, int ban = 0) {
        // rt：根；ban：禁止经过的根相邻点，0 表示无
        int n = (int)g.size() - 1;
        vector<int> f(n + 1, -1), ord{rt};
        vector<H> h(n + 1);
        f[rt] = ban;
        for (int i = 0; i < (int)ord.size(); i++) {
            int u = ord[i];
            for (int v : g[u]) if (v != f[u]) f[v] = u, ord.push_back(v);
        }
        for (int i = (int)ord.size() - 1; i >= 0; i--) {
            int u = ord[i];
            H x{1, 1};
            for (int v : g[u]) if (f[v] == u) {
                x[0] += rnd(h[v][0] + sd[0]);
                x[1] += rnd(h[v][1] + sd[1]);
            }
            h[u] = x;
        }
```

```
        return h;
    }

    H root(const vector<vector<int>> &g, int rt, int ban = 0) {
        return build(g, rt, ban)[rt];
    }

    vector<H> code(const vector<vector<int>> &g) { // 返回无根树规范哈希
        auto c = cen(g);
        if (c.size() == 1) return {root(g, c[0])};
        vector<H> r{root(g, c[0], c[1]), root(g, c[1], c[0])};
        sort(r.begin(), r.end());
        return r;
    }

    bool same(const vector<vector<int>> &a, const vector<vector<int>> &b) {
        // a、b：两棵 1-index 无根树
        return a.size() == b.size() && code(a) == code(b);
    }
};
```

节点带标签时，把 `H x{1,1}` 改为由 `tag[u]` 与两组 `sd` 分别 `rnd` 得到的初值；比较的所有树必须使用相同的标签编码。需要对大量有根子树分类时，以指定根调用一次 `build`，直接使用返回的 `h[u]`，不要为每个点重复遍历。

---

# 11. 欧拉路径 / 欧拉回路

欧拉路径要求每条边恰好使用一次，输出的是点序列。Hierholzer 算法支持重边和自环。

存在条件（忽略零度点后还必须连通）：

- 无向欧拉回路：所有点度数为偶数。
- 无向欧拉路径：恰有 `0` 或 `2` 个奇度点；有两个时必须从奇度点出发。
- 有向欧拉回路：每点入度等于出度。
- 有向欧拉路径：至多一个点 `出度=入度+1` 作为起点，至多一个点 `入度=出度+1` 作为终点，其余相等。
- `Euler eu(n,dir)`：`dir=true` 为有向图，否则为无向图。
- `add(u,v)`：加入一条边。
- `trail(s)`：从 `s` 开始求路径；失败返回空数组，成功返回长度为 `m+1` 的点序列。

复杂度 `O(n+m)`。调用会消耗内部游标，同一对象不要重复调用 `trail`。

下面的辅助函数根据边表自动检查条件，并返回一个可用起点。它只检查“忽略零度点后的弱连通性”；有向图的强连通要求由欧拉条件保证。

```
pair<bool, int> euler_check(int n, const vector<pair<int, int>>& e, bool dir) { //
e: 边表; dir: 是否有向
    DSU d(n);
```

```
    vector<int> in(n + 1), out(n + 1), deg(n + 1);
    for (auto [u, v] : e) {
        d.merge(u, v);
        if (dir) ++out[u], ++in[v];
        else ++deg[u], ++deg[v];
    }
    int s = 1, nz = 0;
    if (dir) {
        int a = 0, b = 0;
        for (int i = 1; i <= n; ++i) {
            if (in[i] + out[i]) {
                if (!nz) s = i;
                ++nz;
            }
            if (out[i] == in[i] + 1) ++a, s = i;
            else if (in[i] == out[i] + 1) ++b;
            else if (in[i] != out[i]) return {false, 0};
        }
        if (!((a == 0 && b == 0) || (a == 1 && b == 1))) return {false, 0};
    } else {
        int odd = 0;
        for (int i = 1; i <= n; ++i) if (deg[i]) {
            if (!nz) s = i;
            ++nz;
            if (deg[i] & 1) ++odd, s = i;
        }
        if (odd != 0 && odd != 2) return {false, 0};
    }
    if (!nz) return {true, 1};
    int rt = d.find(s);
    for (int i = 1; i <= n; ++i) {
        int z = dir ? in[i] + out[i] : deg[i];
        if (z && d.find(i) != rt) return {false, 0};
    }
    return {true, s};
}
```

```
struct Euler {
    int n, m = 0;
    bool dir;
    vector<vector<pair<int, int>>> g;
    vector<int> it;
    vector<char> used;
    Euler(int n, bool dir) : n(n), dir(dir), g(n + 1), it(n + 1) {}
    void add(int u, int v) { // 加 u->v(dir=1) 或 u-v(dir=0)
        int id = m++;
        g[u].push_back({v, id});
        if (!dir) g[v].push_back({u, id});
        used.push_back(0);
    }
    vector<int> trail(int s) { // s: 起点；失败返回空
```

```
        vector<int> st = {s}, ans;
        while (!st.empty()) {
            int u = st.back();
            while (it[u] < (int)g[u].size() && used[g[u][it[u]].second]) ++it[u];
            if (it[u] == (int)g[u].size()) {
                ans.push_back(u), st.pop_back();
            } else {
                auto [v, id] = g[u][it[u]++];
                if (!used[id]) used[id] = 1, st.push_back(v);
            }
        }
        if ((int)ans.size() != m + 1) return {};
        reverse(ans.begin(), ans.end());
        return ans;
    }
};
```

若题目要输出边编号，将栈元素改成“当前点、进入它的边号”，回退时记录边号。要求字典序最小时，先将每个邻接表按目标点逆序排序，让末尾作为栈顶取出。

---

## 12. 二分图最大匹配 Hopcroft-Karp

求无权二分图最大匹配。左部编号 `1..nl`，右部编号 `1..nr`，两侧编号互相独立。

- `HK hk(nl,nr)`：指定左右部大小。
- `add(u,v)`：加入左点 `u` 到右点 `v` 的边。
- `work()`：返回最大匹配边数。
- `ml[u]` 是左点匹配的右点，`mr[v]` 是右点匹配的左点；`0` 表示未匹配。

复杂度 `O(m sqrt(nl+nr))`，空间 `O(n+m)`。

```
struct HK {
    int nl, nr;
    vector<vector<int>> g;
    vector<int> ml, mr, d;
    HK(int nl, int nr) : nl(nl), nr(nr), g(nl + 1),
                         ml(nl + 1), mr(nr + 1), d(nl + 1) {}
    void add(int u, int v) { g[u].push_back(v); } // 左 u - 右 v
    bool bfs() {
        queue<int> q;
        fill(d.begin(), d.end(), -1);
        for (int u = 1; u <= nl; ++u) if (!ml[u]) d[u] = 0, q.push(u);
        bool ok = false;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : g[u]) {
                int w = mr[v];
                if (!w) ok = true;
                else if (d[w] == -1) d[w] = d[u] + 1, q.push(w);
```

```
            }
        }
        return ok;
    }
    bool dfs(int u) {
        for (int v : g[u]) {
            int w = mr[v];
            if (!w || (d[w] == d[u] + 1 && dfs(w))) {
                ml[u] = v, mr[v] = u;
                return true;
            }
        }
        d[u] = -1;
        return false;
    }
    int work() { // 返回最大匹配数
        int ans = 0;
        while (bfs())
            for (int u = 1; u <= nl; ++u) if (!ml[u] && dfs(u)) ++ans;
        return ans;
    }
};
```

常用等价关系（仅对二分图成立）：

- 最小点覆盖大小 = 最大匹配大小。
- 最大独立集大小 = 总点数 - 最大匹配大小。
- DAG 的最小路径覆盖大小 = 点数 - 拆点后二分图最大匹配大小。

---

# 13. 一般图最大匹配 Blossom

求无向一般图的最大基数匹配，允许奇环，不能用二分图匹配。`match[u]` 为点 `u` 匹配的点，返回值为匹配边数。

- `Blossom b(n)`：建立 `1..n`。
- `add(u,v)`：加入无向边。
- `work()`：返回最大匹配数，并填充 `match`。

复杂度 `O(n^3)`，适用于 `n` 为几百量级的题目；大规模一般图匹配通常需要更专门的实现。

```
struct Blossom {
    int n;
    vector<vector<int>> g;
    vector<int> mt, p, base, q;
    vector<char> used, bl;
    Blossom(int n) : n(n), g(n + 1), mt(n + 1), p(n + 1), base(n + 1), q(n + 1),
                     used(n + 1), bl(n + 1) {}
    void add(int u, int v) { g[u].push_back(v), g[v].push_back(u); } // u-v
    int lca(int a, int b) {
```

```
        vector<char> vis(n + 1);
        while (1) {
            a = base[a], vis[a] = 1;
            if (!mt[a]) break;
            a = p[mt[a]];
        }
        while (1) {
            b = base[b];
            if (vis[b]) return b;
            b = p[mt[b]];
        }
    }
    void mark(int v, int b, int c) {
        while (base[v] != b) {
            bl[base[v]] = bl[base[mt[v]]] = 1;
            p[v] = c, c = mt[v], v = p[mt[v]];
        }
    }
    bool bfs(int rt) {
        fill(used.begin(), used.end(), 0);
        fill(p.begin(), p.end(), 0);
        iota(base.begin(), base.end(), 0);
        int l = 0, r = 0;
        q[r++] = rt, used[rt] = 1;
        while (l < r) {
            int u = q[l++];
            for (int v : g[u]) {
                if (base[u] == base[v] || mt[u] == v) continue;
                if (v == rt || (mt[v] && p[mt[v]])) {
                    int b = lca(u, v);
                    fill(bl.begin(), bl.end(), 0);
                    mark(u, b, v), mark(v, b, u);
                    for (int i = 1; i <= n; ++i) if (bl[base[i]]) {
                        base[i] = b;
                        if (!used[i]) used[i] = 1, q[r++] = i;
                    }
                } else if (!p[v]) {
                    p[v] = u;
                    if (!mt[v]) {
                        while (v) {
                            int u2 = p[v], w = mt[u2];
                            mt[v] = u2, mt[u2] = v, v = w;
                        }
                        return true;
                    }
                    v = mt[v], used[v] = 1, q[r++] = v;
                }
            }
        }
        return false;
    }
    int work() { // 返回最大匹配数
```

```
        int ans = 0;
        for (int i = 1; i <= n; ++i) if (!mt[i] && bfs(i)) ++ans;
        return ans;
    }
};
```

## 14. 二分图最大权完美匹配 KM

求 `n*n` 完备权值矩阵上的最大权完美匹配。若左右两侧大小不同，补虚点；若某条边不存在，填一个足够小的负数并根据题意判断是否允许该匹配。

- `KM k(n)`：左右两侧都为 `1..n`。
- `add(u,v,w)`：设置左 `u` 与右 `v` 的权值 `w`；未设置边默认为 `0`。
- `work()`：返回最大总权，并填充 `ml[u]`、`mr[v]`。

复杂度 `O(n^3)`，权值和使用 `ll`。

```
struct KM {
    int n;
    vector<vector<ll>> a;
    vector<ll> lx, ly, sl;
    vector<int> mr;
    vector<char> vx, vy;
    KM(int n) : n(n), a(n + 1, vector<ll>(n + 1)), lx(n + 1), ly(n + 1),
                sl(n + 1), mr(n + 1), vx(n + 1), vy(n + 1) {}
    void add(int u, int v, ll w) { a[u][v] = w; } // 左 u、右 v 的权 w
    bool dfs(int u) {
        vx[u] = 1;
        for (int v = 1; v <= n; ++v) if (!vy[v]) {
            ll d = lx[u] + ly[v] - a[u][v];
            if (!d) {
                vy[v] = 1;
                if (!mr[v] || dfs(mr[v])) return mr[v] = u, true;
            } else sl[v] = min(sl[v], d);
        }
        return false;
    }
    ll work() { // 返回最大权完美匹配值
        for (int i = 1; i <= n; ++i)
            lx[i] = *max_element(a[i].begin() + 1, a[i].end());
        for (int u = 1; u <= n; ++u) {
            fill(sl.begin(), sl.end(), INF);
            while (1) {
                fill(vx.begin(), vx.end(), 0), fill(vy.begin(), vy.end(), 0);
                if (dfs(u)) break;
                ll d = INF;
                for (int v = 1; v <= n; ++v) if (!vy[v]) d = min(d, sl[v]);
                for (int i = 1; i <= n; ++i) if (vx[i]) lx[i] -= d;
                for (int v = 1; v <= n; ++v) {
                    if (vy[v]) ly[v] += d;
                    else sl[v] -= d;
```

```
                }
            }
        }
        vector<int> ml(n + 1);
        for (int v = 1; v <= n; ++v) if (mr[v]) ml[mr[v]] = v;
        ll ans = 0;
        for (int u = 1; u <= n; ++u) ans += a[u][ml[u]];
        return ans;
    }
    vector<int> left_match() { // work 后调用：返回左点匹配的右点
        vector<int> ml(n + 1);
        for (int v = 1; v <= n; ++v) if (mr[v]) ml[mr[v]] = v;
        return ml;
    }
};
```

若要求最小权，把所有权值取相反数后调用 `work()`，最后将结果取相反数。

## 15. 最大流 Dinic

求有向容量网络的最大流，也等价于指定源汇的最小割。无向容量边不是简单调用两次 `add` 的所有场景都等价，必须按题意建模。

- `Dinic f(n)`：建立 `1..n`。
- `add(u,v,c)`：加容量为 `c` 的有向边及容量为 `0` 的反向残量边。
- `flow(s,t,lim)`：求 `s` 到 `t` 的最大流，最多增广到 `lim`；默认不限制。
- 调用后，从 `s` 在残量网络沿正容量边可达的点构成最小割的源点侧。

容量和答案使用 `ll`。复杂度一般很快；二分图单位容量网络复杂度更优。

```
struct Dinic {
    struct E { int v, r; ll c; };
    int n;
    vector<vector<E>> g;
    vector<int> lv, it;
    Dinic(int n) : n(n), g(n + 1), lv(n + 1), it(n + 1) {}
    void add(int u, int v, ll c) { // u->v, 容量 c
        int a = g[u].size(), b = g[v].size();
        g[u].push_back({v, b, c});
        g[v].push_back({u, a, 0});
    }
    bool bfs(int s, int t) {
        fill(lv.begin(), lv.end(), -1);
        queue<int> q;
        lv[s] = 0, q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (auto& e : g[u]) if (e.c > 0 && lv[e.v] == -1)
                lv[e.v] = lv[u] + 1, q.push(e.v);
        }
```

```
        return lv[t] != -1;
    }
    ll dfs(int u, int t, ll f) {
        if (u == t) return f;
        for (int& i = it[u]; i < (int)g[u].size(); ++i) {
            E& e = g[u][i];
            if (e.c <= 0 || lv[e.v] != lv[u] + 1) continue;
            ll x = dfs(e.v, t, min(f, e.c));
            if (x) {
                e.c -= x, g[e.v][e.r].c += x;
                return x;
            }
        }
        return 0;
    }
    ll flow(int s, int t, ll lim = INF) { // s,t: 源汇; lim: 流量上限
        ll ans = 0;
        while (ans < lim && bfs(s, t)) {
            fill(it.begin(), it.end(), 0);
            while (ll x = dfs(s, t, lim - ans)) ans += x;
        }
        return ans;
    }
};
```

典型建模：

- 二分图匹配：`S -> 左部 -> 右部 -> T`，容量均为 `1`。
- 点容量/删点代价：把点 `u` 拆成 `uin -> uout`，该边容量为点容量，其余边接在两侧。
- 多源多汇：增加超级源、超级汇，以足够大的容量连接。

`INF` 容量不应盲目取极限值，通常取“所有有限容量之和 + 1”最稳妥。

---

# 16. 最小费用最大流

每条边同时有容量和单位流量费用。模板先用 SPFA 求初始势能，因此允许负费用边，但要求从源点可达的残量网络中不存在负费用环。

- `MCMF f(n)`。
- `add(u,v,cap,cost)`：加容量 `cap`、单位费用 `cost` 的有向边。
- `flow(s,t,lim)`：返回 `{实际流量,最小总费用}`；若无法送满 `lim`，第一项会小于 `lim`。

每轮使用势能 Dijkstra，复杂度约 `O(Fm log n)`，其中 `F` 是增广轮数相关的总流量；容量很大但路径每次能推很多流时会明显更快。

```
struct MCMF {
    struct E { int v, r; ll c, w; };
    int n;
    vector<vector<E>> g;
```

```cpp
    MCMF(int n) : n(n), g(n + 1) {}
    void add(int u, int v, ll c, ll w) { // u->v, 容量 c, 费用 w
        int a = g[u].size(), b = g[v].size();
        g[u].push_back({v, b, c, w});
        g[v].push_back({u, a, 0, -w});
    }
    pair<ll, ll> flow(int s, int t, ll lim = INF) { // 返回 {流量,费用}
        vector<ll> h(n + 1, INF), d(n + 1);
        vector<int> pv(n + 1), pe(n + 1);
        vector<char> in(n + 1);
        queue<int> q;
        h[s] = 0, q.push(s), in[s] = 1;
        while (!q.empty()) {
            int u = q.front(); q.pop(), in[u] = 0;
            for (auto& e : g[u]) if (e.c && h[e.v] > h[u] + e.w) {
                h[e.v] = h[u] + e.w;
                if (!in[e.v]) in[e.v] = 1, q.push(e.v);
            }
        }
        for (int i = 1; i <= n; ++i) if (h[i] == INF) h[i] = 0;
        ll fl = 0, co = 0;
        while (fl < lim) {
            fill(d.begin(), d.end(), INF);
            priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll,
int>>> pq;
            d[s] = 0, pq.push({0, s});
            while (!pq.empty()) {
                auto [du, u] = pq.top(); pq.pop();
                if (du != d[u]) continue;
                for (int i = 0; i < (int)g[u].size(); ++i) {
                    E& e = g[u][i];
                    ll nd = du + e.w + h[u] - h[e.v];
                    if (e.c && d[e.v] > nd) {
                        d[e.v] = nd, pv[e.v] = u, pe[e.v] = i;
                        pq.push({nd, e.v});
                    }
                }
            }
            if (d[t] == INF) break;
            for (int i = 1; i <= n; ++i) if (d[i] != INF) h[i] += d[i];
            ll x = lim - fl;
            for (int v = t; v != s; v = pv[v]) x = min(x, g[pv[v]][pe[v]].c);
            for (int v = t; v != s; v = pv[v]) {
                E& e = g[pv[v]][pe[v]];
                co += x * e.w, e.c -= x, g[v][e.r].c += x;
            }
            fl += x;
        }
        return {fl, co};
    }
};
```

求最大费用时可把所有费用取相反数，最后将答案费用再取反。注意 `流量 * 费用` 也必须在 `ll` 范围内。

---

## 17. 2-SAT

有 `n` 个布尔变量，每个约束是两个文字的或：`(x_u=a) OR (x_v=b)`。把它转成两条蕴含边：`(x_u!=a) -> (x_v=b)` 与 `(x_v!=b) -> (x_u=a)`，再求 SCC。

- 变量编号从 `0` 到 `n-1`。
- `add(u,a,v,b)`：加入 `(x[u]==a) OR (x[v]==b)`。
- `force(u,a)`：强制 `x[u]=a`。
- `work()`：有解返回 `true`，无解返回 `false`。
- `ans[u]` 是一组可行赋值。

复杂度 `O(n+m)`。

```
struct TwoSAT {
    int n, tim = 0, cnt = 0;
    vector<vector<int>> g;
    vector<int> dfn, low, stk, id, ans;
    vector<char> ins;
    TwoSAT(int n) : n(n), g(2 * n), dfn(2 * n), low(2 * n),
                    id(2 * n, -1), ans(n), ins(2 * n) {}
    int p(int u, bool a) { return 2 * u + a; } // 文字 x[u]=a
    void imp(int u, bool a, int v, bool b) { g[p(u, a)].push_back(p(v, b)); }
    void add(int u, bool a, int v, bool b) { // (x[u]=a) OR (x[v]=b)
        imp(u, !a, v, b), imp(v, !b, u, a);
    }
    void force(int u, bool a) { add(u, a, u, a); } // 强制 x[u]=a
    void dfs(int u) {
        dfn[u] = low[u] = ++tim;
        stk.push_back(u), ins[u] = 1;
        for (int v : g[u]) {
            if (!dfn[v]) dfs(v), low[u] = min(low[u], low[v]);
            else if (ins[v]) low[u] = min(low[u], dfn[v]);
        }
        if (dfn[u] != low[u]) return;
        while (1) {
            int v = stk.back(); stk.pop_back();
            ins[v] = 0, id[v] = cnt;
            if (v == u) break;
        }
        ++cnt;
    }
    bool work() { // 返回是否有解，答案在 ans
        for (int i = 0; i < 2 * n; ++i) if (!dfn[i]) dfs(i);
        for (int i = 0; i < n; ++i) {
            if (id[p(i, 0)] == id[p(i, 1)]) return false;
            ans[i] = id[p(i, 0)] > id[p(i, 1)];
```

```
        }
        return true;
    }
};
```

常见约束写法：

| 约束 | 调用 |
| --- | --- |
| `A or B` | `add(A.u,A.val,B.u,B.val)` |
| `A -> B` | `add(A.u,!A.val,B.u,B.val)` |
| `A == B` | 加 `A->B` 和 `B->A`，并补上两条逆否命题 |
| `A != B` | `add(A,B)` 且 `add(!A,!B)` |
| 至少一个为真 | `add(u,1,v,1)` |
| 至多一个为真 | `add(u,0,v,0)` |

这里的 `A` 表示一个完整文字，例如 `x[u]=true`，不是单纯的变量编号。

---

# 18. 无向图全局最小割 Stoer-Wagner

不指定源汇，求删除最小总边权后使无向图不连通的代价。要求边权非负，支持重边。

- `w[u][v]` 为无向边权矩阵；重边累加。
- `stoer(w)` 返回全局最小割。
- 点编号使用 `0..n-1`，这是本板子中少数 0-index 模板。

复杂度 `O(n^3)`，空间 `O(n^2)`，适合点数较小的稠密图。原图已经不连通时答案为 `0`。

```
ll stoer(vector<vector<ll>> w) { // w: 0-index 无向边权矩阵
    int n = w.size();
    if (n < 2) return 0;
    vector<int> v(n);
    iota(v.begin(), v.end(), 0);
    ll ans = INF;
    while (v.size() > 1) {
        vector<ll> d(n);
        vector<char> vis(n);
        int pre = -1, sel = -1;
        for (int i = 0; i < (int)v.size(); ++i) {
            sel = -1;
            for (int x : v) if (!vis[x] && (sel == -1 || d[x] > d[sel])) sel = x;
            if (sel == -1) return ans;
            if (i + 1 == (int)v.size()) {
                ans = min(ans, d[sel]);
```

```
                for (int x : v) if (x != sel) w[pre][x] += w[sel][x], w[x][pre] =
w[pre][x];
                v.erase(find(v.begin(), v.end(), sel));
            } else {
                vis[sel] = 1, pre = sel;
                for (int x : v) if (!vis[x]) d[x] += w[sel][x];
            }
        }
    }
    return ans;
}
```

如果需要输出割的具体点集，要在每次合并时额外维护超级点包含的原始点，并在刷新最优答案时保存最后加入阶段的集合。

---

# 19. Prüfer 序列

Prüfer 序列在标号树和树的计数题中很常见。编码长度为 `n-2`，解码后得到一棵 `n` 个点的树。

- `encode(g)`：输入 `1..n` 的树邻接表，返回 Prüfer 序列。
- `decode(c)`：输入序列，返回 `n-1` 条无向边；其中 `n=c.size()+2`。
- 两个过程均支持 `n=1/2` 的边界（空序列分别对应单点树/一条边）。

```
vector<int> prufer_encode(const vector<vector<int>>& g) { // g: 1-index 树
    int n = (int)g.size() - 1;
    vector<int> d(n + 1);
    priority_queue<int, vector<int>, greater<int>> q;
    for (int i = 1; i <= n; ++i) {
        d[i] = g[i].size();
        if (d[i] == 1) q.push(i);
    }
    vector<int> c;
    for (int i = 0; i < n - 2; ++i) {
        int x = q.top();
        q.pop();
        int y = 0;
        for (int z : g[x]) if (d[z]) { y = z; break; }
        c.push_back(y);
        if (--d[y] == 1) q.push(y);
        d[x] = 0;
    }
    return c;
}

vector<pair<int, int>> prufer_decode(const vector<int>& c, int n) { // c: Prüfer 序
列；显式 n 支持 n=1
    if (n <= 1) return {};
    assert((int)c.size() == n - 2);
    vector<int> d(n + 1, 1);
```

```
    for (int x : c) ++d[x];
    priority_queue<int, vector<int>, greater<int>> q;
    for (int i = 1; i <= n; ++i) if (d[i] == 1) q.push(i);
    vector<pair<int, int>> e;
    for (int x : c) {
        int y = q.top(); q.pop();
        e.push_back({y, x});
        if (--d[x] == 1) q.push(x);
    }
    if (n >= 2) {
        int x = q.top(); q.pop();
        int y = q.top();
        e.push_back({x, y});
    }
    return e;
}

vector<pair<int, int>> prufer_decode(const vector<int>& c) { // c: Prüfer 序列；默认
n=|c|+2
    return prufer_decode(c, (int)c.size() + 2);
}
```

## 19.1 树的计数结论

标号完全图上的树数为 `n^(n-2)`（Cayley 公式）。若一个森林有 `k` 个连通块，块大小为 `s_i`，补 `k-1` 条边变成树的方案数为 `n^(k-2) * product(s_i)`；`k=1` 时答案为 `1`。

## 19.2 矩阵树定理 Kirchhoff

矩阵树定理统计图的生成树数，也支持重边和带权边。无向图拉普拉斯矩阵满足 `L[u][u]+=w`、`L[v][v]+=w`、`L[u][v]-=w`、`L[v][u]-=w`；删除任意一行一列后的行列式就是带权生成树和。

- `span_tree(n,e,mod)`：无向边 `e[i]={u,v,w}`，返回生成树权值乘积之和。
- `arb_tree(n,e,rt,out,mod)`：有向边同格式；`out=true` 统计从根 `rt` 指向所有点的外向树，`false` 统计所有点指向根的内向树。

代码要求 `mod` 为质数。复杂度 $O(n^3)$，空间 $O(n^2)$；自环没有贡献，重边直接重复加入或合并权值。无权图令每条边 `w=1`。

```
ll det_mod(vector<vector<ll>> a, ll mod) { // a：方阵；mod：质数
    int n = a.size();
    ll r = 1;
    auto pw = [&](ll x, ll y) {
        ll z = 1;
        for (; y; y >>= 1, x = x * x % mod) if (y & 1) z = z * x % mod;
        return z;
    };
    for (int i = 0; i < n; i++) {
        int p = i;
        while (p < n && !a[p][i]) p++;
        if (p == n) return 0;
```

```
        if (p != i) swap(a[p], a[i]), r = r ? mod - r : 0;
        r = r * a[i][i] % mod;
        ll iv = pw(a[i][i], mod - 2);
        for (int j = i + 1; j < n; j++) if (a[j][i]) {
            ll q = a[j][i] * iv % mod;
            for (int k = i; k < n; k++)
                a[j][k] = (a[j][k] - q * a[i][k] % mod + mod) % mod;
        }
    }
    return r;
}

ll span_tree(int n, const vector<tuple<int, int, ll>> &e, ll mod) {
    // n：点数；e：无向边 {u,v,w}；mod：质数
    if (n == 1) return 1 % mod;
    vector<vector<ll>> a(n, vector<ll>(n));
    for (auto [u, v, w] : e) if (u != v) {
        --u, --v; w = (w % mod + mod) % mod;
        a[u][u] = (a[u][u] + w) % mod, a[v][v] = (a[v][v] + w) % mod;
        a[u][v] = (a[u][v] - w + mod) % mod;
        a[v][u] = (a[v][u] - w + mod) % mod;
    }
    vector<vector<ll>> b(n - 1, vector<ll>(n - 1));
    for (int i = 0; i < n - 1; i++) for (int j = 0; j < n - 1; j++) b[i][j] = a[i]
[j];
    return det_mod(move(b), mod);
}

ll arb_tree(int n, const vector<tuple<int, int, ll>> &e,
            int rt, bool out, ll mod) {
    // rt：根；out：1 为根指向外；其余参数同 span_tree
    if (n == 1) return 1 % mod;
    vector<vector<ll>> a(n + 1, vector<ll>(n + 1));
    for (auto [u, v, w] : e) if (u != v) {
        w = (w % mod + mod) % mod;
        if (out) a[v][v] = (a[v][v] + w) % mod,
                 a[v][u] = (a[v][u] - w + mod) % mod;
        else a[u][u] = (a[u][u] + w) % mod,
             a[u][v] = (a[u][v] - w + mod) % mod;
    }
    vector<vector<ll>> b(n - 1, vector<ll>(n - 1));
    for (int i = 1, x = 0; i <= n; i++) if (i != rt) {
        for (int j = 1, y = 0; j <= n; j++) if (j != rt) b[x][y++] = a[i][j];
        x++;
    }
    return det_mod(move(b), mod);
}
```

## 20. 环检测与最小环

## 20.1 输出有向图任意一个环

返回的序列首尾相同，内部点按环上顺序排列；返回空数组表示无环。支持自环和二元环。

```
vector<int> find_dir_cycle(const vector<vector<int>>& g) { // g: 1-index 有向图
    int n = (int)g.size() - 1;
    vector<char> vis(n + 1);
    vector<int> fa(n + 1), ans;
    function<bool(int)> dfs = [&](int u) {
        vis[u] = 1;
        for (int v : g[u]) {
            if (!vis[v]) {
                fa[v] = u;
                if (dfs(v)) return true;
            } else if (vis[v] == 1) {
                vector<int> t;
                for (int x = u;; x = fa[x]) {
                    t.push_back(x);
                    if (x == v) break;
                }
                reverse(t.begin(), t.end()), t.push_back(v), ans = move(t);
                return true;
            }
        }
        vis[u] = 2;
        return false;
    };
    for (int i = 1; i <= n; ++i) if (!vis[i] && dfs(i)) break;
    return ans;
}
```

## 20.2 输出无向图任意一个环

参数邻接表中的第二项是边编号；同一条无向边的两个方向必须使用同一个编号。返回序列首尾相同，支持重边与自环。

```
vector<int> find_undir_cycle(const vector<vector<pair<int, int>>>& g) { // g: {终点, 边号}
    int n = (int)g.size() - 1;
    vector<char> vis(n + 1);
    vector<int> fa(n + 1), fe(n + 1), ans;
    function<bool(int, int)> dfs = [&](int u, int pe) {
        vis[u] = 1;
        for (auto [v, id] : g[u]) {
            if (id == pe) continue;
            if (!vis[v]) {
                fa[v] = u, fe[v] = id;
                if (dfs(v, id)) return true;
            } else if (vis[v] == 1) {
                vector<int> t;
                for (int x = u;; x = fa[x]) {
```

```
                    t.push_back(x);
                    if (x == v) break;
                }
                reverse(t.begin(), t.end()), t.push_back(v), ans = move(t);
                return true;
            }
        }
        vis[u] = 2;
        return false;
    };
    for (int i = 1; i <= n; ++i) if (!vis[i] && dfs(i, -1)) break;
    return ans;
}
```

## 20.3 无向无权图最小环

`g` 使用 `{终点,边号}` 的无向邻接表。返回最小环边数；无环返回 `INF`。复杂度 `O(n(n+m))`，适合中小规模。

```
int min_cycle(const vector<vector<pair<int, int>>>& g) { // g: 无向无权图，边号成对相同
    int n = (int)g.size() - 1, ans = INT_MAX;
    for (int s = 1; s <= n; ++s) {
        vector<int> d(n + 1, -1), pe(n + 1, -1);
        queue<int> q;
        d[s] = 0, q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (auto [v, id] : g[u]) {
                if (id == pe[u]) continue;
                if (d[v] == -1) d[v] = d[u] + 1, pe[v] = id, q.push(v);
                else ans = min(ans, d[u] + d[v] + 1);
            }
        }
    }
    return ans;
}
```

## 20.4 本质不同简单环计数（状压）

仅适合 `n<=20` 的无向简单图。每个无向简单环只计一次，环套环分别计数。复杂度 `O(m 2^n)`，空间 `O(n 2^n)`。

```
ll count_cycles(const vector<vector<int>>& g) { // g: 0-index 无向简单图
    int n = (int)g.size();
    vector<vector<ll>> dp(1 << n, vector<ll>(n));
    for (int i = 0; i < n; ++i) dp[1 << i][i] = 1;
    ll ans = 0;
    for (int st = 1; st < (1 << n); ++st) {
        int low = __builtin_ctz((unsigned)st), start = 1 << low;
        for (int u = 0; u < n; ++u) if (dp[st][u]) {
```

```
            for (int v : g[u]) {
                if ((1 << v) < start) continue;
                if (st >> v & 1) {
                    if (v == low) ans += dp[st][u];
                } else dp[st | (1 << v)][v] += dp[st][u];
            }
        }
    }
    int m = 0;
    for (auto& a : g) m += a.size();
    m /= 2;
    return (ans - m) / 2;
}
```

# 21. 常见图论结论与建模

## 连通性与环

1. 有向图是否为 DAG：拓扑排序后得到的点数是否为 `n`。
2. 无向图是否有环：逐边用 DSU 合并，若某条边两端已经连通则出现环；重边和自环按题意处理。
3. 有向图中，一个点能无限行走，当且仅当它能到达某个环。可先求 SCC：大小大于 `1` 的 SCC 有环；大小为 `1` 时还需检查自环，再在反图上扩展。
4. 有向完全图（竞赛图）只要有有向环，就一定有三元环。
5. 简单平面图且 `n>=3` 时有 `m<=3n-6`；简单二分平面图有 `m<=2n-4`。这些只是必要条件，不是充分条件。

## 树与生成树

1. 一棵 `n` 个点的树恰有 `n-1` 条边；无向图满足"连通且 `m=n-1`"即可判定为树。
2. 任意树都是二分图。二分染色两侧大小为 `a,b`，在不破坏二分图性质时最多还能加入 `a*b-(n-1)` 条简单边。
3. 树边 `(u,v)` 删掉后两侧点数为 `s` 与 `n-s`，所有点对路径经过该边的对数为 `s*(n-s)`。
4. 标号点 `1..n` 能形成的不同树有 `n^(n-2)` 棵（Cayley 公式）。
5. 一个森林有 `k` 个连通块、各块大小为 `s_i`，添加 `k-1` 条边使其成为树的方案数为 `n^(k-2) * product(s_i)`；`k=1` 时为 `1`。
6. 树直径：任选点 BFS/DFS 找最远点 `a`，再从 `a` 找最远点 `b`，`dist(a,b)` 为直径。该方法要求树边权非负；有负权时要用树形 DP。

## SCC、桥与割

1. SCC 缩点后必为 DAG。
2. 使非强连通有向图变强连通，最少加边数为缩点 DAG 中 `max(零入度点数, 零出度点数)`。
3. 无向连通图删桥缩点后得到树；使其变为边双连通图，最少加边数为 `ceil(叶子数/2)`。
4. 删除最少边使指定源汇不连通是最小割；删除点则拆点，把点代价放在 `uin -> uout` 的容量上。

## 二分图与匹配

1. 无向图是二分图，当且仅当不存在奇环。
2. 二分图最大匹配数 = 最小点覆盖大小（Kőnig 定理）。
3. 二分图最大独立集大小 = 总点数 - 最大匹配数。
4. DAG 最小路径覆盖大小 = 点数 - 拆点后二分图最大匹配数。

## 差分约束

约束 `x_v - x_u <= w` 建边 `u -> v`，边权 `w`，求一组可行解等价于求最短路；存在负环表示无解。若所有变量都没有指定源点，建立超级源向每个点连权 `0` 的边。

约束方向速记：

- `x_v <= x_u + w`：`u -> v (w)`。
- `x_v - x_u >= w`：改写成 `x_u - x_v <= -w`，建 `v -> u (-w)`。
- `x_u == x_v`：同时加入 `x_u-x_v<=0` 和 `x_v-x_u<=0`。

---

# 22. 赛场检查清单

1. 先确认点编号是 0-index 还是 1-index；本板子除 Stoer-Wagner 外均按说明使用 1-index，2-SAT 的变量为 0-index。
2. 确认边是有向还是无向。无向边通常要加两次，但 DSU、Kruskal 的边表只存一次。
3. 权值、距离、答案、容量、费用使用 `ll`；点号、边号、度数使用 `int`。
4. 重边：Floyd 取最小，Stoer-Wagner/流网络通常累加，桥必须按边编号跳父边。
5. 自环：不会进入 MST，不是桥；在单点 SCC 中可以构成有向环。
6. 图可能不连通时，DFS/BFS/Tarjan 要从所有未访问点启动。
7. Dijkstra 禁止负边；0-1 BFS 只允许 `0/1` 边；DAG DP 前必须判环。
8. 恢复路径或方案时，松弛/匹配/增广处同步记录前驱，不要只保存答案值。
9. 多测时重新构造结构体最省事，避免 `dfn`、度数、残量网络、当前弧没有清空。
10. 递归 Tarjan、Dinic DFS 在极深图上可能爆栈，提交前结合平台栈限制和 `n` 判断。